

```
notebook_name = "Lab12.2_Transformers_TextClassification_YourName_RollNo.ipynb"
```

```
# Data handling
import pandas as pd
import numpy as np

# Dataset loading and processing
from datasets import load_dataset

# Model building (Transformer model)
from transformers import BertTokenizer, BertForSequenceClassification

# Training utilities
from transformers import Trainer, TrainingArguments

# Evaluation metrics
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

# PyTorch (Deep Learning backend)
import torch
```

```
# Data handling
import pandas as pd
import numpy as np

# Dataset loading and processing
from datasets import load_dataset

# Model building (Transformer model)
from transformers import BertTokenizer, BertForSequenceClassification

# Training utilities
from transformers import Trainer, TrainingArguments

# Evaluation metrics
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

# PyTorch (Deep Learning backend)
import torch
```

```
import matplotlib.pyplot as plt

# Load the dataset (using 'imdb' as a placeholder for text classification)
dataset = load_dataset("imdb")

# Convert to pandas for analysis
train_df = pd.DataFrame(dataset["train"])

# Count class distribution
class_counts = train_df["label"].value_counts()

print(class_counts)

# Plot distribution
class_counts.plot(kind='bar')
plt.title("Class Distribution")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md:      7.81k/? [00:00<00:00, 597kB/s]

plain_text/train-00000-of-00001.parquet: 100%                21.0M/21.0M [00:01<00:00, 12.3MB/s]

plain_text/test-00000-of-00001.parquet: 100%                20.5M/20.5M [00:00<00:00, 27.9MB/s]

plain_text/unsupervised-00000-of-00001.p(...): 100%         42.0M/42.0M [00:01<00:00, 38.7MB/s]

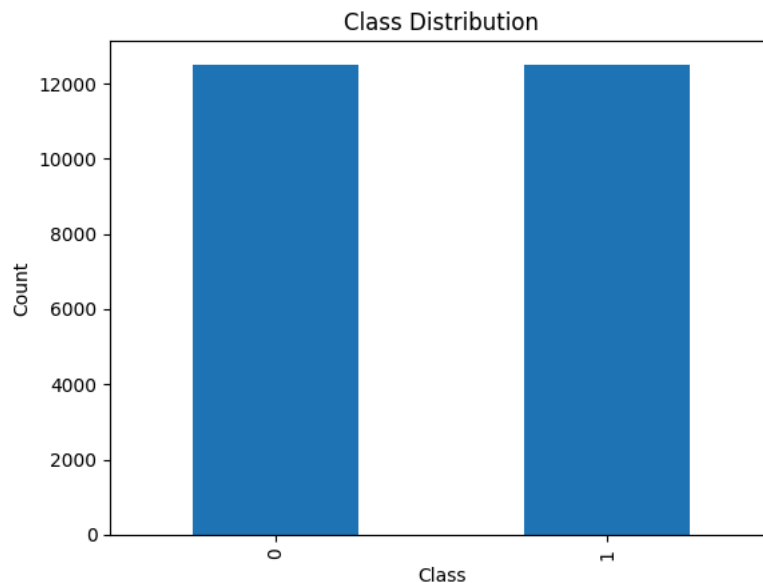
Generating train split: 100%                                25000/25000 [00:00<00:00, 93888.21 examples/s]

Generating test split: 100%                                25000/25000 [00:00<00:00, 176439.85 examples/s]

Generating unsupervised split: 100%                        50000/50000 [00:00<00:00, 148933.10 examples/s]

label
0      12500
1      12500
Name: count, dtype: int64

```



```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
tokenizer_config.json: 100%                                48.0/48.0 [00:00<00:00, 5.02kB/s]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
vocab.txt: 100%                                           232k/232k [00:00<00:00, 644kB/s]
```

```
tokenizer.json: 100%                                     466k/466k [00:00<00:00, 905kB/s]
```

```

def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",      # Pad to fixed length
        truncation=True,          # Truncate long sequences
        max_length=128            # Fixed sequence length
    )

tokenized_datasets = dataset.map(tokenize_function, batched=True)

```

```
Map: 100%                                25000/25000 [00:18<00:00, 1306.44 examples/s]
```

```
Map: 100%                                25000/25000 [00:18<00:00, 1531.11 examples/s]
```

```
Map: 100%                                50000/50000 [00:36<00:00, 1328.85 examples/s]
```

```
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
```

```
tokenized_datasets.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
```

```

train_dataset = tokenized_datasets["train"]
test_dataset = tokenized_datasets["test"]

```

```
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

config.json: 100%

570/570 [00:00<00:00, 54.2kB/s]

model.safetensors: 100%

440M/440M [00:06<00:00, 85.5MB/s]

Loading weights: 100%

199/199 [00:00<00:00, 588.51it/s, Materializing param=bert.pooler.dense.weight]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task

```
from transformers import TrainingArguments
```

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",      # Evaluate at end of each epoch
    save_strategy="epoch",
    learning_rate=2e-5,        # Fine-tuning learning rate
    per_device_train_batch_size=16, # Batch size for training
    per_device_eval_batch_size=16,  # Batch size for evaluation
    num_train_epochs=3,          # Number of training epochs
    weight_decay=0.01,
    logging_dir="./logs",
    load_best_model_at_end=True
)
```

`logging\_dir` is deprecated and will be removed in v5.2. Please set `TensorBoard` instead.

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average="binary"
    )
    acc = accuracy_score(labels, preds)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```

```
from transformers import Trainer
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```